上海交通大學
SHANGHAI JIAO TONG UNIVERSITY



Data Mining and Big Data Analysis

Chapter16: Social Networks and Community Detection

社会网络和社区发现

丁 玥

Spring 2025

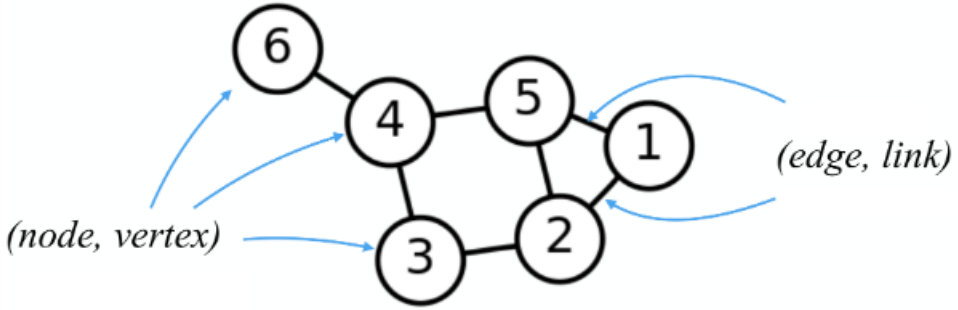
社会网络和社区发现

- **社会网络**（Social Network）
  - —是由许多节点以及节点间关系构成的一个网络结构
  - —节点通常是指个人或组织（又称社团）
  - —社会网络代表各种社会关系，产生的网络结构往往是非常复杂的



社会网络和社区发现

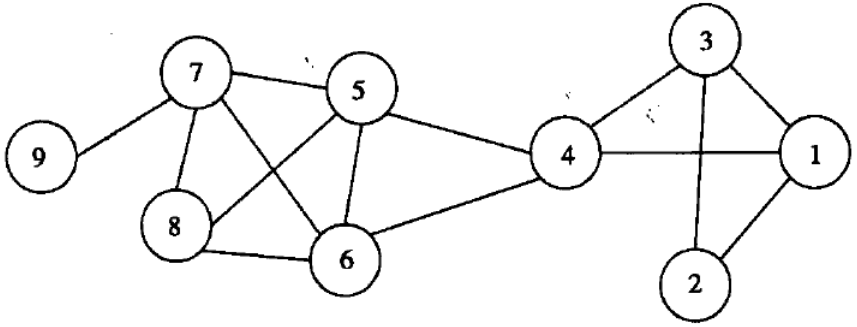
- 社会网络的表示：基于图的方法
  - —结点表示个人或组织
  - —边用来连接结点表示各种关系，比如朋友关系、亲属关系等。





社会网络和社区发现

- 社会网络也可以表示为一个矩阵，称为社会矩阵(social matrix)或邻接矩阵（adjacency matrix）

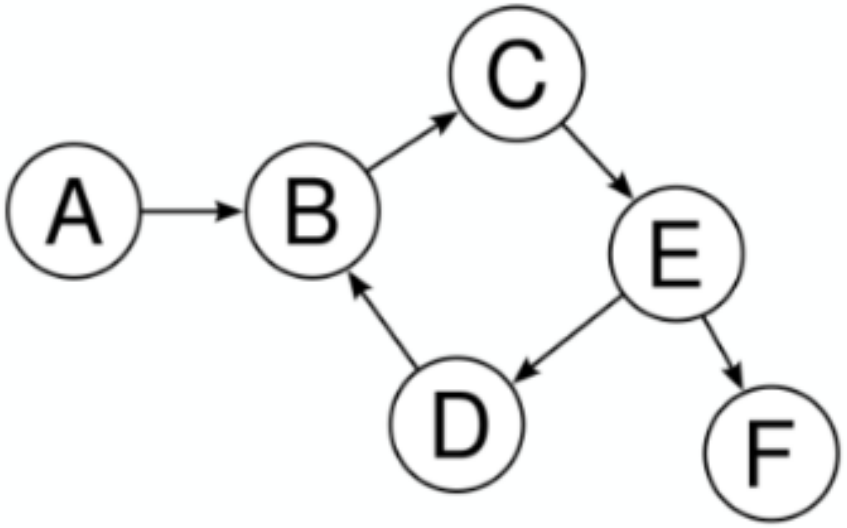


结点	1	2	3	4	5	6	7	8	9
1	—	1	1	1	0	0	0	0	0
2	1	—	1	0	0	0	0	0	0
3	1	1	—	1	0	0	0	0	0
4	1	0	1	—	1	1	0	0	0
5	0	0	0	1	—	1	1	1	0
6	0	0	0	1	1	—	1	1	0
7	0	0	0	0	1	1	—	1	1
8	0	0	0	0	1	1	1	—	0
9	0	0	0	0	0	0	1	0	—

网络是无向的，
与此相对应的邻接矩阵是对称的

社会网络和社区发现

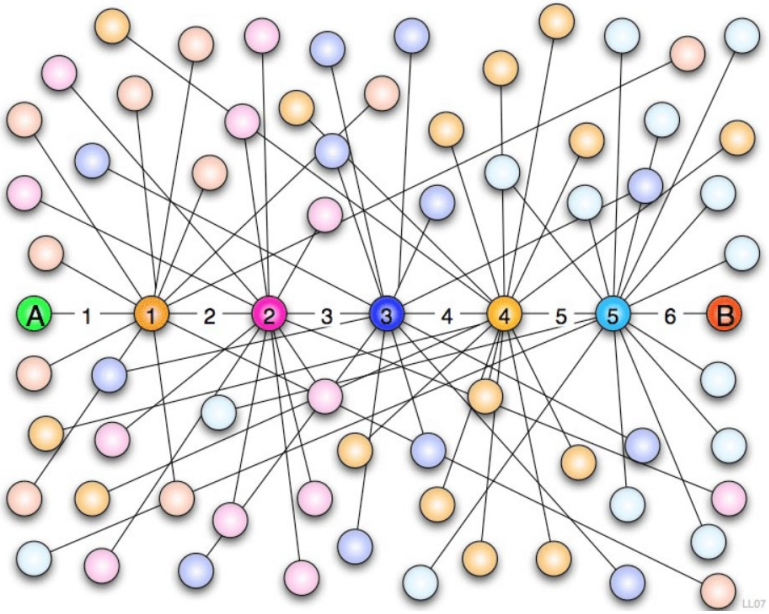
- 在一些社会媒体站点中， 交互是有方向的。比如在Twitter 中，一个用户x关注(follow)用户y，但y不一定关注x。在这种情况下网络是有向的， 也是不对称的



	A	B	C	D	E	F
A	0	1	0	0	0	0
B	0	0	1	0	0	0
C	0	0	0	0	1	0
D	0	1	0	0	0	0
E	0	0	0	1	0	1
F	0	0	0	0	0	0

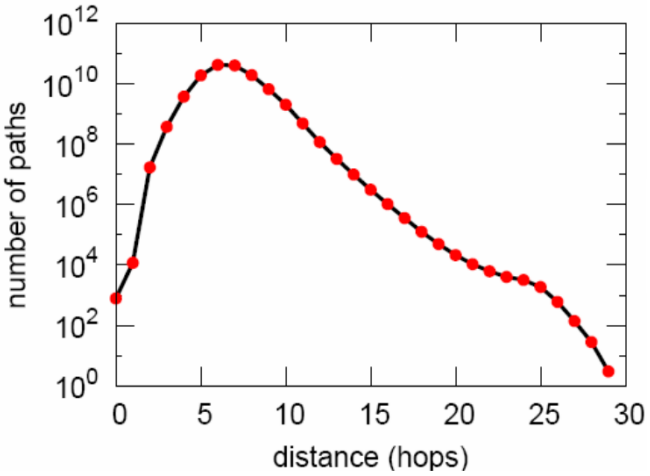
社会网络和社区发现

- 社会网络的另一个显著特性就是小世界效应
  - Travers和Milgram(1969)做了一个实验来检验美国人在社会网络中平均路径的长度。实验通过熟人之间传递信件的方式把信件转发给陌生人。
  - 实验最终的结果表明平均路径长度是5.5, 接近6 。也就是说，最多通过六个人你就能够认识任何一个陌生人， 这就是著名的六度分离(six degrees of separation)理论



社会网络和社区发现

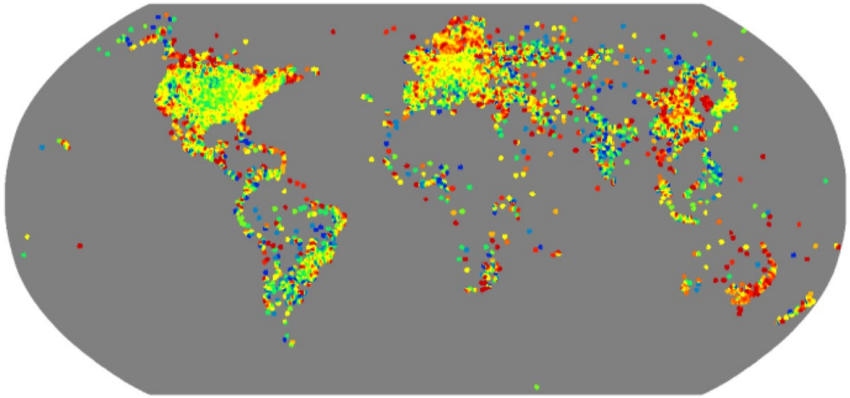
- 这个结论也被一个巨大的即时信息网络所证实
  - J. Leskovec and E. Horvitz. Planetary-Scale Views on a Large Instant-Messaging Network, *Proceedings of WWW 2008*, Beijing, China, April 2008.
  - 在MSN Messenger系统中获取全球匿名通信模式的快照
  - 包含 1.8 亿个节点和 13 亿条无向边，是当时最大的社交网络
  - 发现 2006 年 7 月期间 Messenger 用户之间通信的平均路径长度为 6.6。



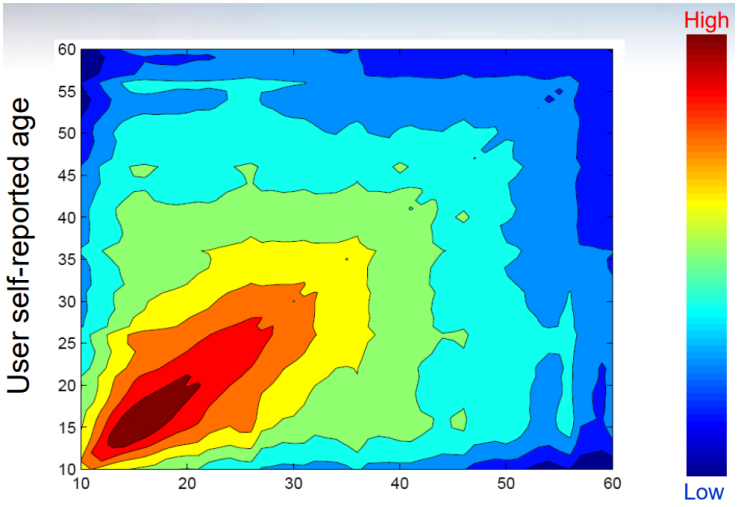
Avg. path length 6.6

社会网络和社区发现

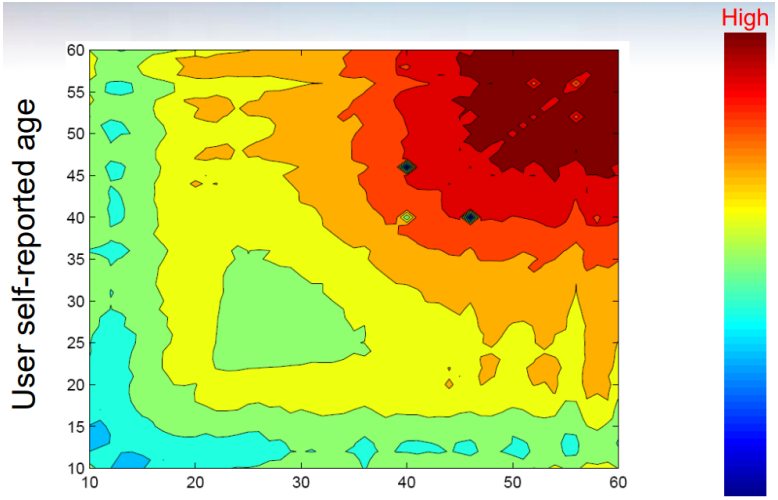
其他有趣的发现



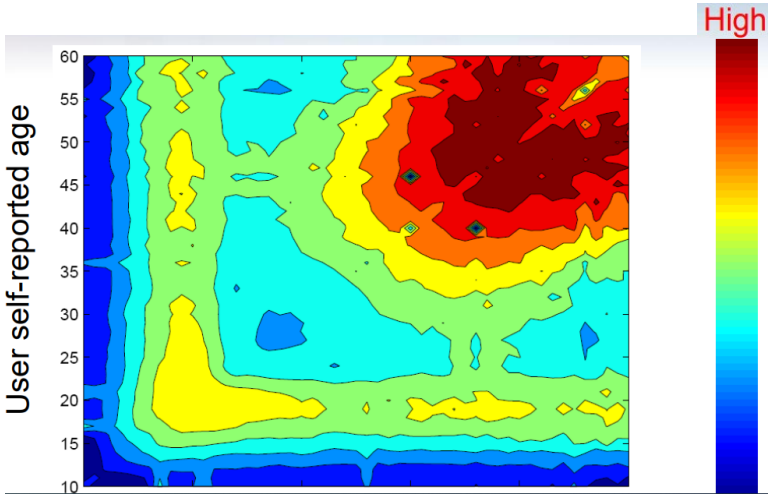
Communication Density



Young people communicate with same age



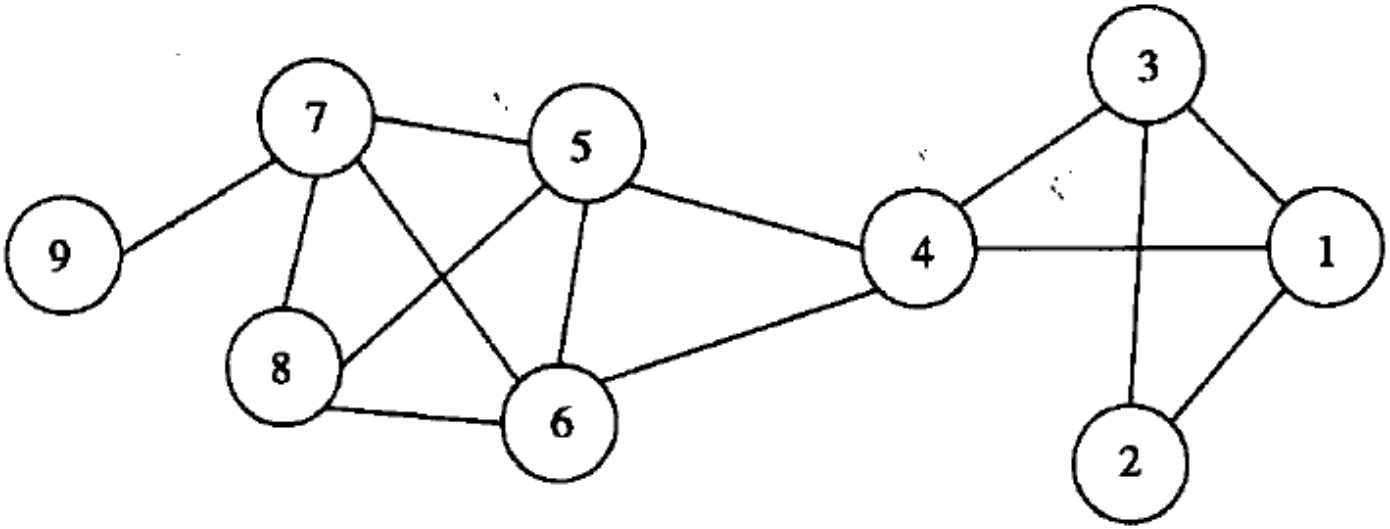
Older people have longer conversations



Older people exchange more messages per session

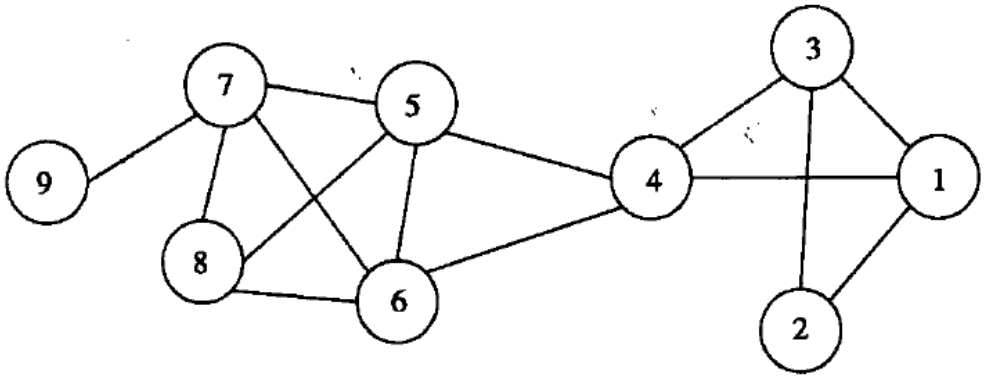
社会网络和社区发现

- 为了规范小世界性的效果， 人们还定义了其他的测量方法。 网络中最长的最短路径就是它的直径(diameter)



社会网络和社区发现

- 社会网络还展现了很强的社区结构（community structure）
  - —也就是人们更倾向于与一个团体中的人交往，而与团体外的人的交往较少
  - —由于朋友的朋友很容易成为朋友，这个传递性可以通过聚类系数（clustering coefficient）来度量，也就是朋友之间的联系（connection）数量与所有联系数量的比例。假设节点v_i有d_i个邻居，在这些邻居之间有k_i条边，那么聚类系数C_i为:



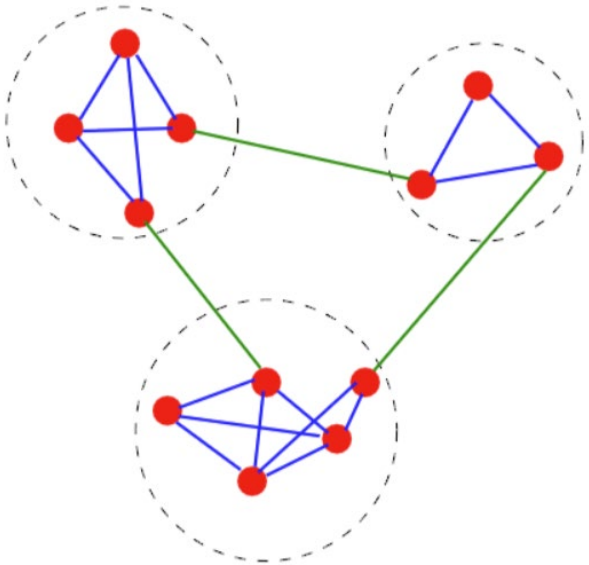
$$C_i = \begin{cases} \dfrac{k_i}{d_i \times (d_i - 1)/2} & d_i > 1 \\ 0 & d_i = 0、1 \end{cases}$$

节点6的聚类系数是多少？

K=4,d=4, C=4/(4*3/2) = 2/3

社会网络和社区发现

- 社区发现
  - 在不同的背景下，社区也可以称为群组 (groups) 、簇 (clusters) 、内聚的子群(cohesive subgroups) ，或模块 (modules)
  - 社区发现是社会网络分析的一个基本任务。发现一个社区就是识别一个结点的集合，使得集合内结点之间的相互作用比它们与集合外结点的相互作用更强



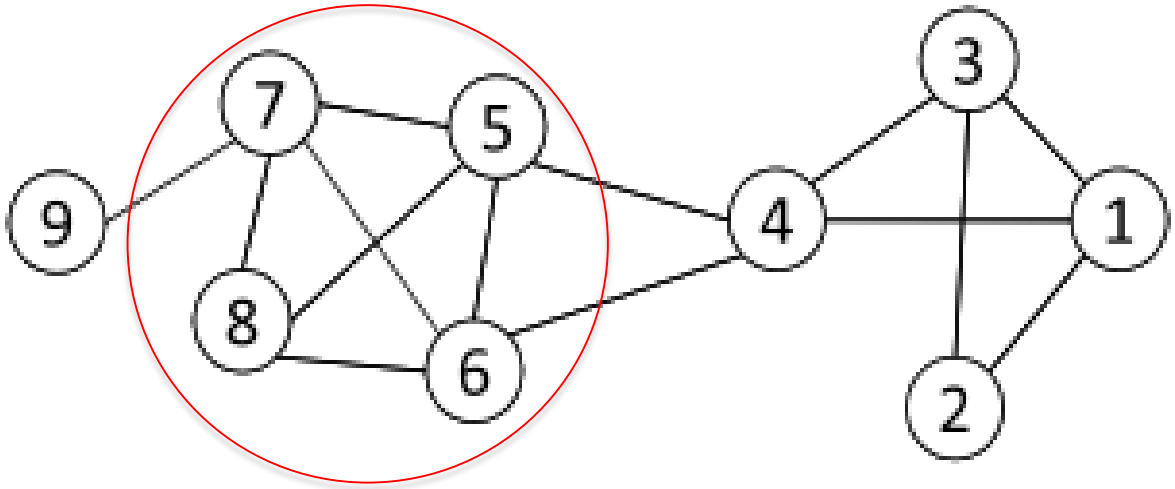
社会网络和社区发现

- 四类社区发现的方法：
  - Node-Centric Community 以节点为中心
  - Group-Centric Community 以群组为中心
  - Network-Centric Community 以网络为中心
  - Hierarchy-Centric Community 以层次结构为中心

社会网络和社区发现

以节点为中心的社区发现

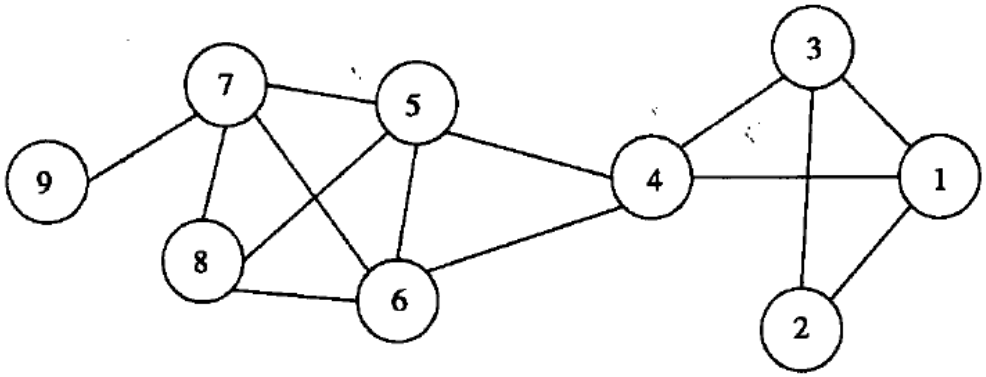
- 团（clique）是图论中的用语。对于给定图G=(V,E)。其中，V={1,…,n}是图G的顶点集，E是图G的边集。图G的团就是一个两两之间有边的顶点集合。简单地说，团是G的一个完全子图
- 一般来说，团的规模越大越有意义，但搜索图中的最大团(maximum clique)是一个NP难问题



{5,6,7,8}是团，包含4个节点

社会网络和社区发现

- **蛮力搜索：遍历网络的所有结点，对于每个结点，检查是否包含那个结点的团**



假设从结点4开始，B_1={4}。对于结点4的每一个下标比它大的邻居，就得到两个规模为2的团：{4, 5}和{4, 6}, 然后将其加入队列。

假设将团B_2={4, 5}从队列中取出，那么最后加入的元素是结点5。可以根据结点5的邻居扩展这个集合，得到3个候选集：{4, 5, 6}、{4, 5, 7}和{4, 5, 8}。但只有{4, 5, 6}是一个团，因为结点6既与结点4连接，又与结点5连接。这样，将{4, 5, 6}加入到队列以便进一步搜索更大的团。

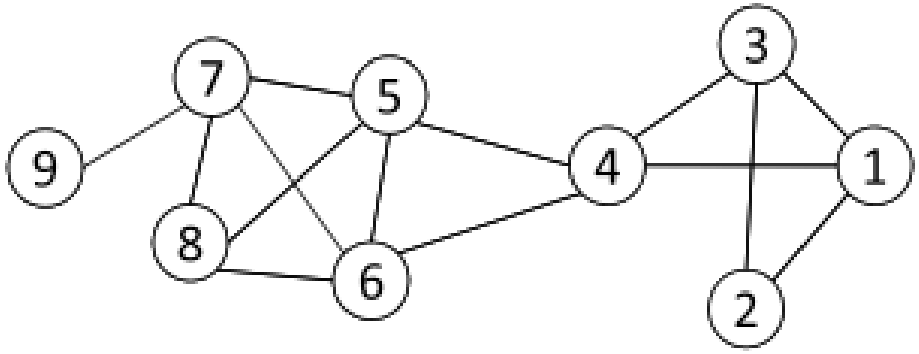
社会网络和社区发现

- 蛮力搜索只能适用于小规模网络，对于大规模网络无能为力。如果要发现一个最大团，一个有效的方法是剪枝策略：剪去那些不太可能包含在团中的结点和边。

- 团过滤算法 Clique Percolation Method (CPM)给定一个用户指定的参数k, 其工作过程如下：
  - 给定网络中找出所有规模为K的团
  - 构建一个团图，如果两个团共享k-1个结点，那么它们是邻接的
  - 在团所构成的图(clique graph)中，每一个连接的部分(component)就是一个社区。

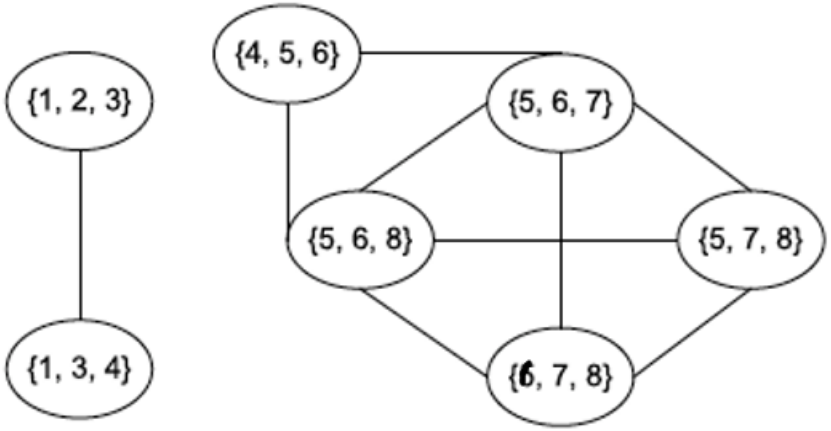
社会网络和社区发现

团过滤算法CPM



给定**k=3**，得到规模为**3**的团
{1, 2, 3}, {1, 3, 4}, {4, 5, 6},
{5, 6, 7}, {5, 6, 8}, {5, 7, 8},
{6, 7, 8}

只要两个团共享k-1个结点，那么他们就被连接起来，每个被连接起来的团的结点都属于同一个社区，因此，得到两个社区{1，2，3，4}和{4，5，6，7，8}



社区:
{1, 2, 3, 4}
{4, 5, 6, 7, 8}

社会网络和社区发现

- 团过滤方法需要枚举所有给定规模为K的团
  - 从计算复杂性来说，这种方法对于大规模网络是不可行的。针对大规模网络，需要采取其他的方法
  - 使用与团相近似的子图来获取社区的结构

社会网络和社区发现

- 以群组为中心的社区发现
  - 在以群组为中心的标准中，将一个群组内的所有链接(link)作为整体考虑。只要这个群组整体上满足一定的要求，就可以允许群组的某些结点具有低的连通性。
  - 其中的一个例子就是基于密度的分组(density-based groups)

社会网络和社区发现

- 以群组为中心的社区发现
  - 定义子图$G_s(V_s, E_s)$是一个准团，如果满足

$$\frac{|E_s|}{|V_s|(|V_s|-1)/2} \geq \gamma$$

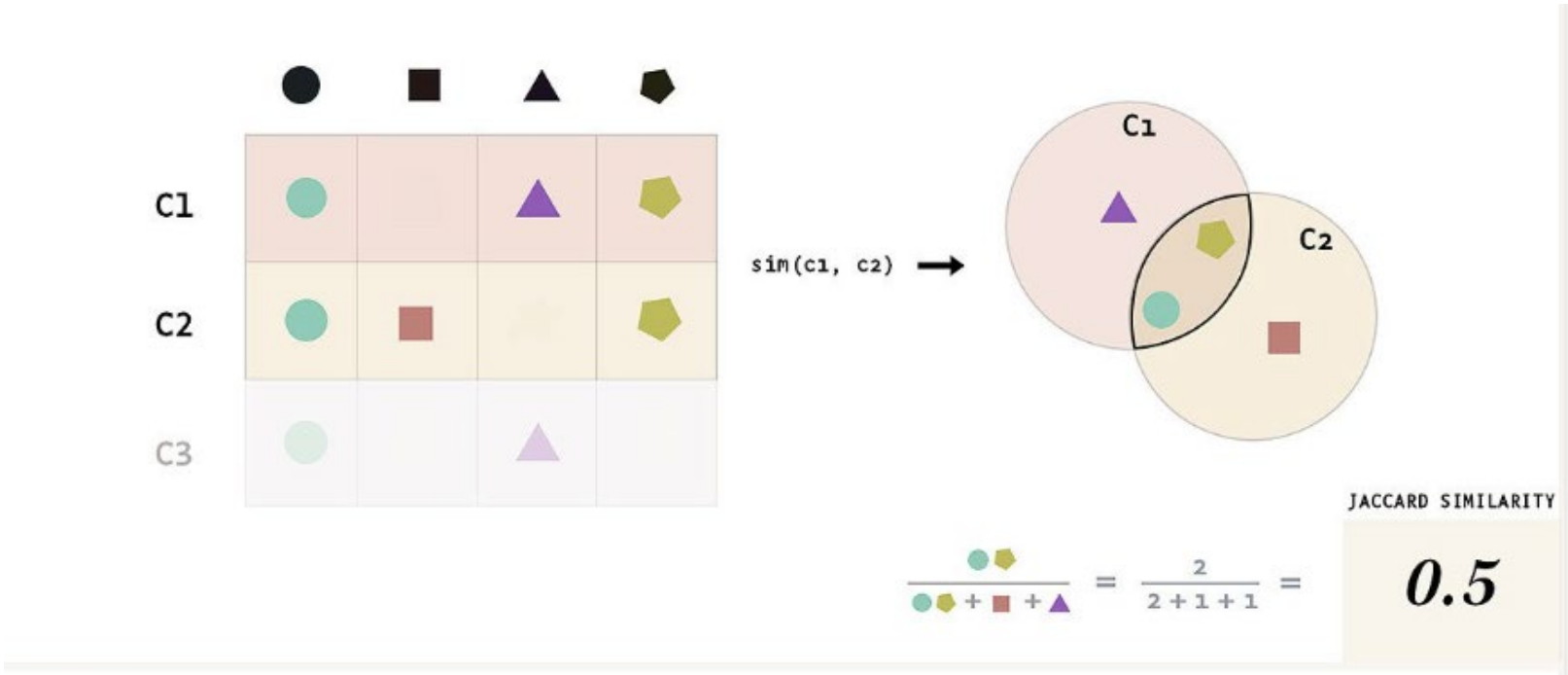
  - 如果$\gamma = 1$，则准团变为团
  - 在网络中搜索最大γ密度的准团，采用与搜索团类似的策略（局部搜索+启发式剪枝）

社会网络和社区发现

- 以网络为中心的社区发现
  - 以网络为中心的社区发现必须考虑网络的整体拓扑结构(global topology)。其目的在于将网络的结点划分为不相交的集合。一般地说，以网络为中心的社区发现试图优化定义于整个网络的标准，而不是优化定义于一个群组的标准
  - 基于相似性度量，传统的K均值聚类(k-means clustering)或者层次聚类算法(hierarchical clustering algorithm)就可以用来发现社区

社会网络和社区发现

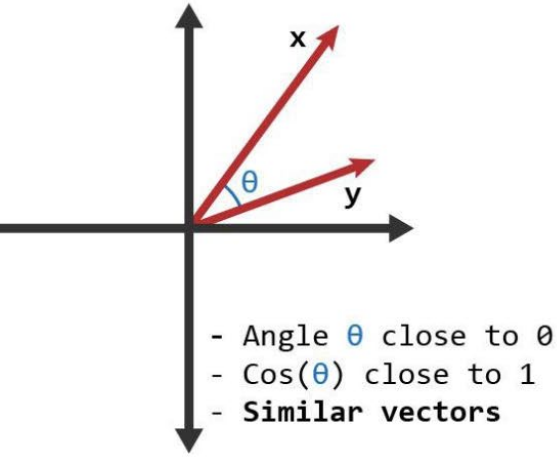
Jaccard Similarity



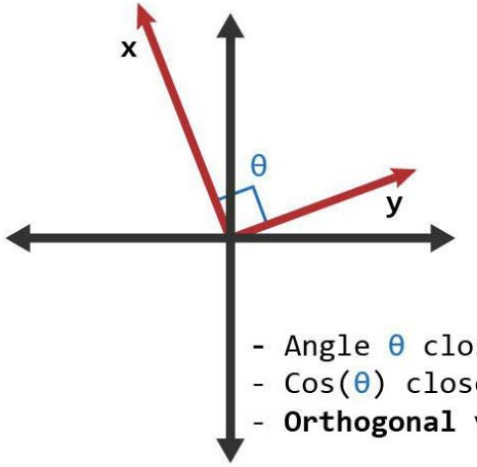
社会网络和社区发现

Cosine Similarity

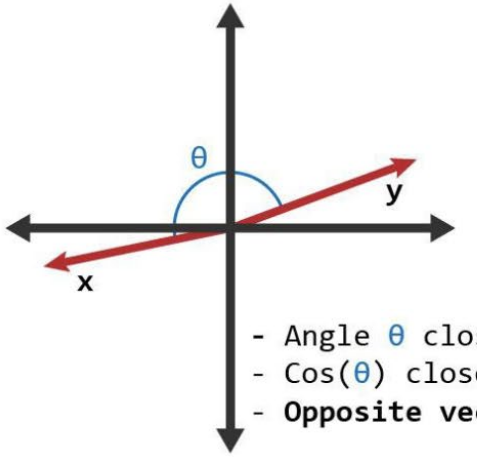
$$similarity(A, B) = cos(\theta) = \frac{A \cdot B}{\|A\|\|B\|}$$



- Angle θ close to 0
- Cos(θ) close to 1
- **Similar vectors**



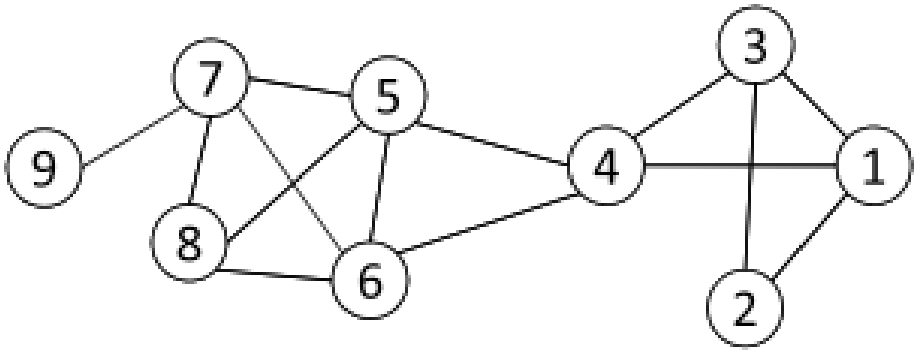
- Angle θ close to 90
- Cos(θ) close to 0
- **Orthogonal vectors**



- Angle θ close to 180
- Cos(θ) close to -1
- **Opposite vectors**

社会网络和社区发现

- 以网络为中心的社区发现
  - —$N_4 = \{1, 3, 5, 6\}$，$N_6 = \{4, 5, 7, 8\}$，相似度为



$$Jaccard(4,6) = \frac{|\{5\}|}{|\{1,3,4,5,6,7,8\}|} = \frac{1}{7}$$

$$cosine(4,6) = \frac{1}{\sqrt{4 \cdot 4}} = \frac{1}{4}$$

社会网络和社区发现

- 然而，根据这种定义，两个相邻的结点之间的相似性可能为0。比如，结点7与结点9, 即使它们相连，它们的相似性也为0
  - $N_7 = \{5, 6, 8, 9\}, N_9 = \{7\}$

- 在计算N_v时，包含结点v
  - 即把网络的邻接矩阵对角线元素设为1，而不是默认的0
  - 这种情况下， $N_7 = \{5, 6, 7, 8, 9\}, N_9 = \{7, 9\}$

$$Jaccard\ (7,9) = \frac{|\{7,9\}|}{|\{5,6,7,8,9\}|} = \frac{2}{5}$$

$$Cosine\ (7,9) = \frac{|\{7,9\}|}{\sqrt{2 \cdot 5}} = \frac{2}{\sqrt{10}}$$

社会网络和社区发现

- 以网络为中心的社区发现
  - 隐含空间模型，将网络的结点映射到一个低维的欧氏空间，使得在新空间中可以保持网络结点的相似性
  - 对于低维空间中的结点，可以采用类似K均值的算法进行聚类

社会网络和社区发现

- 以层次为中心的社区发现
  - 分裂式层次聚类(divisive hierarchical clustering)
  - 聚合式层次聚类(agglomerative hierarchical clustering)

社会网络和社区发现

- 分裂式层次聚类的一般步骤

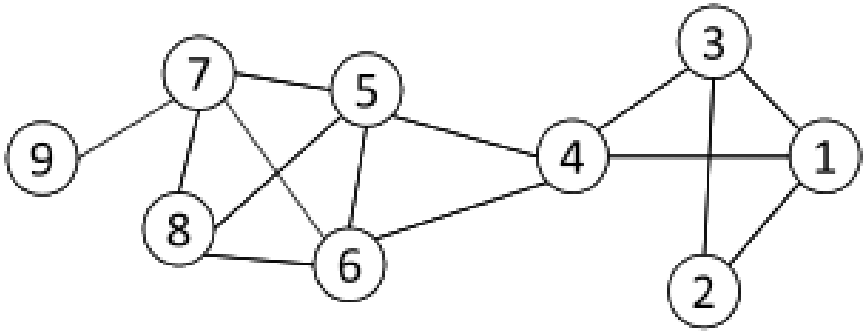
  （1）在每次迭代中，找到最小强度的边，这类边很可能就是连接两个社区之间的边

  （2）删除那条边，然后更新各条边的连接强度

  （3）一旦网络被划分为两个连接的部分，每一部分就认为是一个社区。重复以上过程就可以发现更小的社区

社会网络和社区发现

- 基于边介数(edge betweenness)发现最弱连接的方法
  - —一条边的边介数定义为网络中最短路径经过这条边的数量
  - —边介数的计算（Brandes 2001, Newman and Girvan, 2004 ）

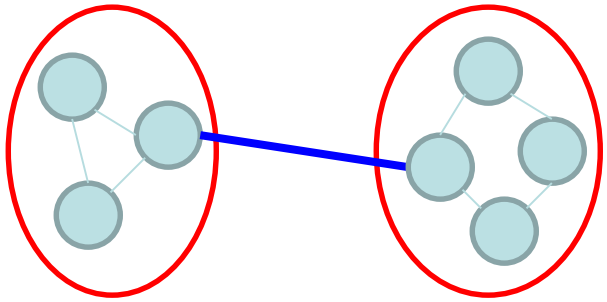


e(1, 2) 的边介数是 **4**
--从结点**2** 到结点 **{4, 5, 6, 7, 8, 9}** 的所有最短路径必须通过**e(1, 2)** 或者**e(2, 3)**

边**e(1,2)**的权重为**6*1/2=3**，同时**e(1,2)** 是结点**1**和**2**的最短路径，因此**e(1,2)**的边介数为**3+1=4**

社会网络和社区发现

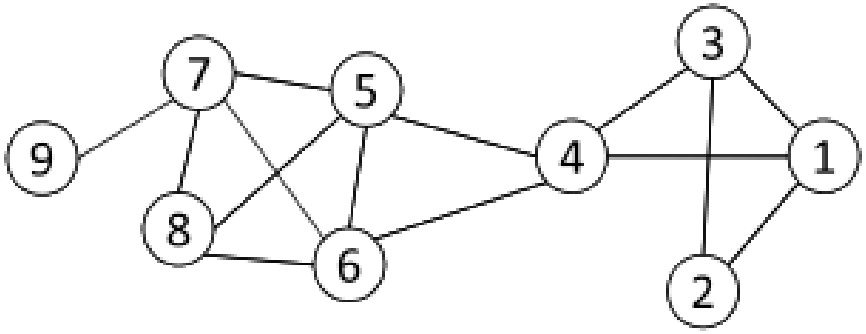
- 基于边介数(edge betweenness)发现最弱连接的方法
  - 如果两个社区只是由少数几条边连接，那么这两个社区之间的路径都要经过这几条边中的一条，所以这几条边的边介数就会很大



  - 自然地，移除边介数大的边就可以发现社区

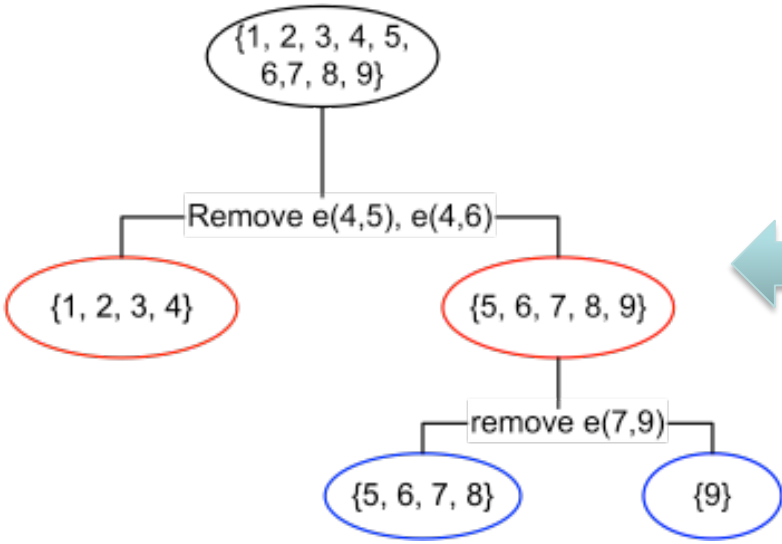
社会网络和社区发现

分裂式层次聚类示例



初始边介数

	1	2	3	4	5	6	7	8	9
1	0	4	1	9	0	0	0	0	0
2	4	0	4	0	0	0	0	0	0
3	1	4	0	9	0	0	0	0	0
4	9	0	9	0	10	10	0	0	0
5	0	0	0	10	0	1	6	3	0
6	0	0	0	10	1	0	6	3	0
7	0	0	0	0	6	6	0	2	8
8	0	0	0	0	3	3	2	0	0
9	0	0	0	0	0	0	8	0	0

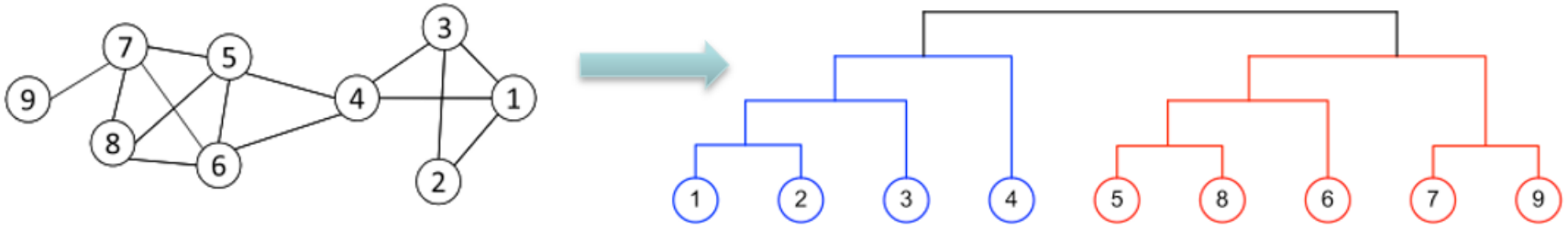


边e(4,5)的边介数为10(最高),移除
边e(4,6)的边介数变为20(最高),移除

边e(7,9)有最高的边介数4，移除

社会网络和社区发现

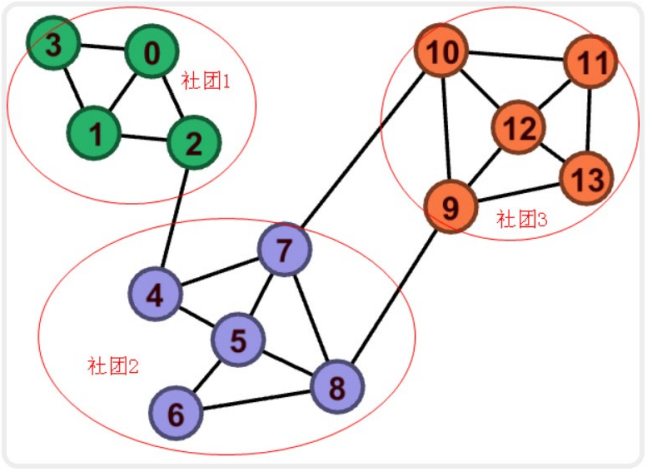
- **聚合式层次聚类**
  - 从基本的社区开始，按照某种标准不断将其聚合成更大的社区
  - 如果两个社区的合并能够获得更高的整体模块度，那么就将其合并
  - 开始时将每个结点视为一个社区，然后不断将他们合并



社会网络和社区发现

聚合式层次聚类

- 模块度（Modularity）是一种评估社区网络结构的指标，用来评估社区内部连接相对于社区之间连接的紧密程度
- 通常模块度为0.3以上表示社区划分质量较为合适



13个结点，23条边，
分成3个社区

模块度（第一版）

$$Q=\sum_i(e_{ii}-a_i^2)=\sum_i e_{ii}-\sum_i a_i^2=Tre-||e^2||$$

在相同社区内结点之间的边集合

所有连接到社区i的边数量

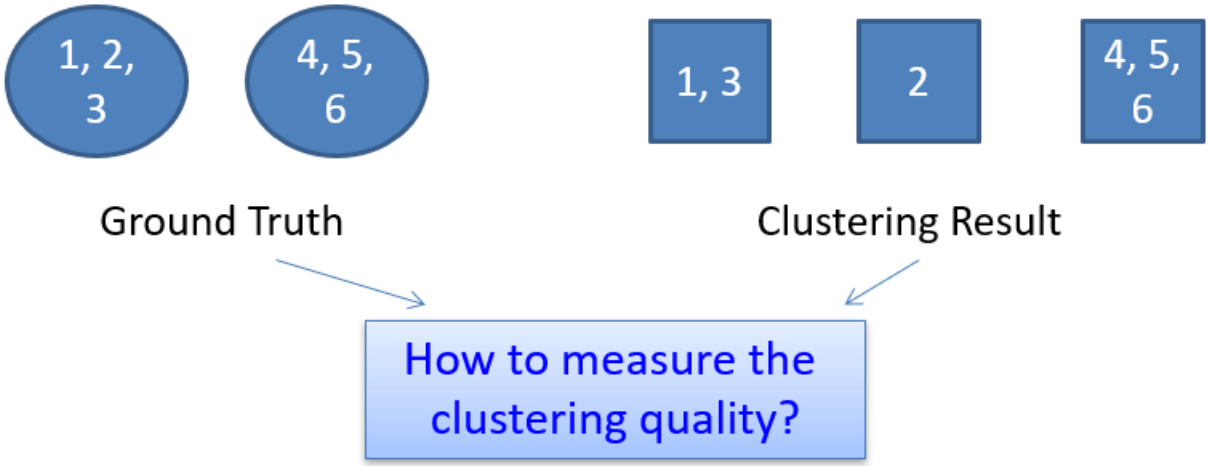
	1	2	3
1	$\frac{5}{23}$	$\frac{1}{2}\times\frac{1}{23}$	$\frac{1}{2}\times\frac{0}{23}$
2	$\frac{1}{2}\times\frac{1}{23}$	$\frac{7}{23}$	$\frac{1}{2}\times\frac{2}{23}$
3	$\frac{1}{2}\times\frac{0}{23}$	$\frac{1}{2}\times\frac{2}{23}$	$\frac{8}{23}$

a	
1	$\frac{5}{23}+\frac{1}{2}\times\frac{1}{23}+\frac{1}{2}\times\frac{0}{23}=\frac{11}{46}$
2	$\frac{1}{2}\times\frac{1}{23}+\frac{7}{23}+\frac{1}{2}\times\frac{2}{23}=\frac{17}{46}$
3	$\frac{1}{2}\times\frac{0}{23}+\frac{1}{2}\times\frac{2}{23}+\frac{8}{23}=\frac{18}{46}$

$$Q=\sum_i(\frac{e_i}{m}-(\frac{k_{C_i}}{2m})^2)=\frac{5}{23}+\frac{7}{23}+\frac{8}{23}-(\frac{11}{46})^2-(\frac{17}{46})^2-(\frac{18}{46})^2=\frac{553}{1058}=0.52268431$$

社区发现的评估

- 评估聚类结果



社区发现的评估

- **For groups with clear definitions**
  - —E.g., Cliques
  - —Verify whether extracted communities satisfy the definition
- **For networks with ground truth information**
  - —Normalized Mutual Information （NMI）
  - —Accuracy of pairwise community memberships

社区发现的评估

- **Entropy:**

$$H(X) = \sum_{x \in X} p(x) \log p(x)$$

- **Mutual Information**

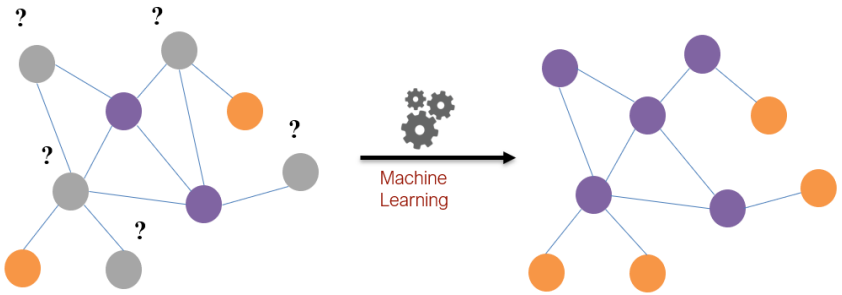
$$I(X;Y) = \sum_{y \in Y} \sum_{x \in X} p(x,y) \log \left(\frac{p(x,y)}{p_1(x) p_2(y)} \right)$$

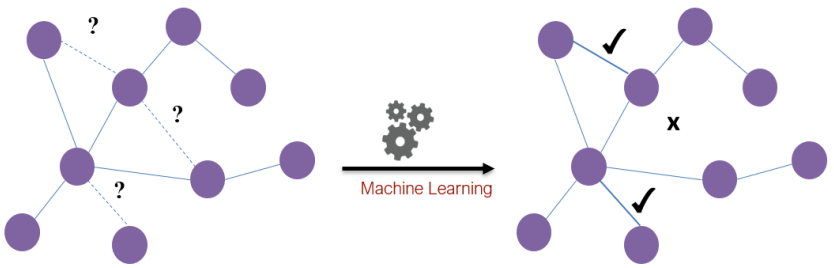
- **Normalized Mutual Information (between 0 and 1)**

$$NMI(X;Y) = \frac{I(X;Y)}{\sqrt{H(X)H(Y)}}$$

图上的数据挖掘——图表示学习

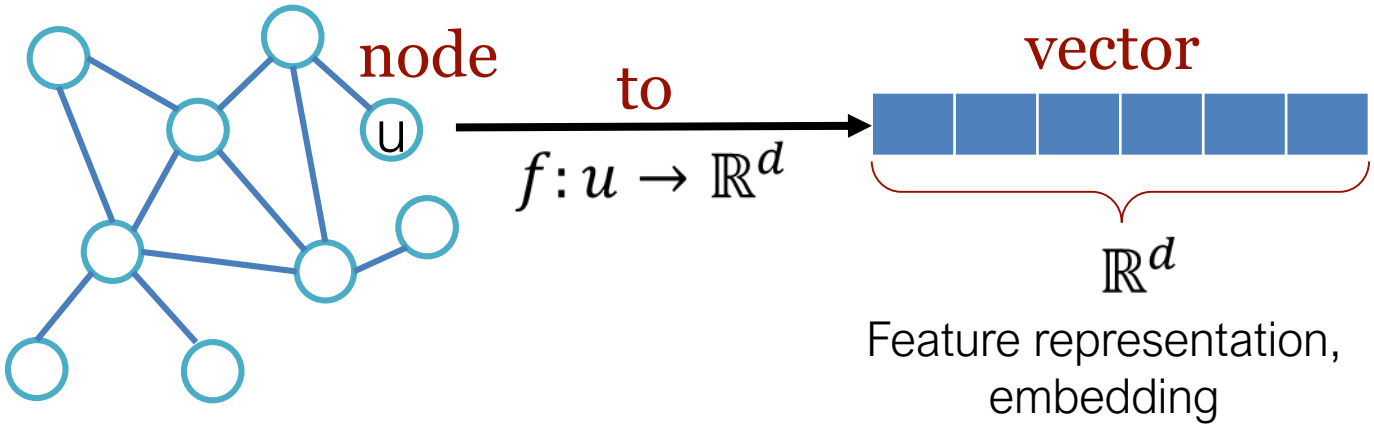
- Community detection
- Node classification
- Link prediction
- Network similarity
- ……





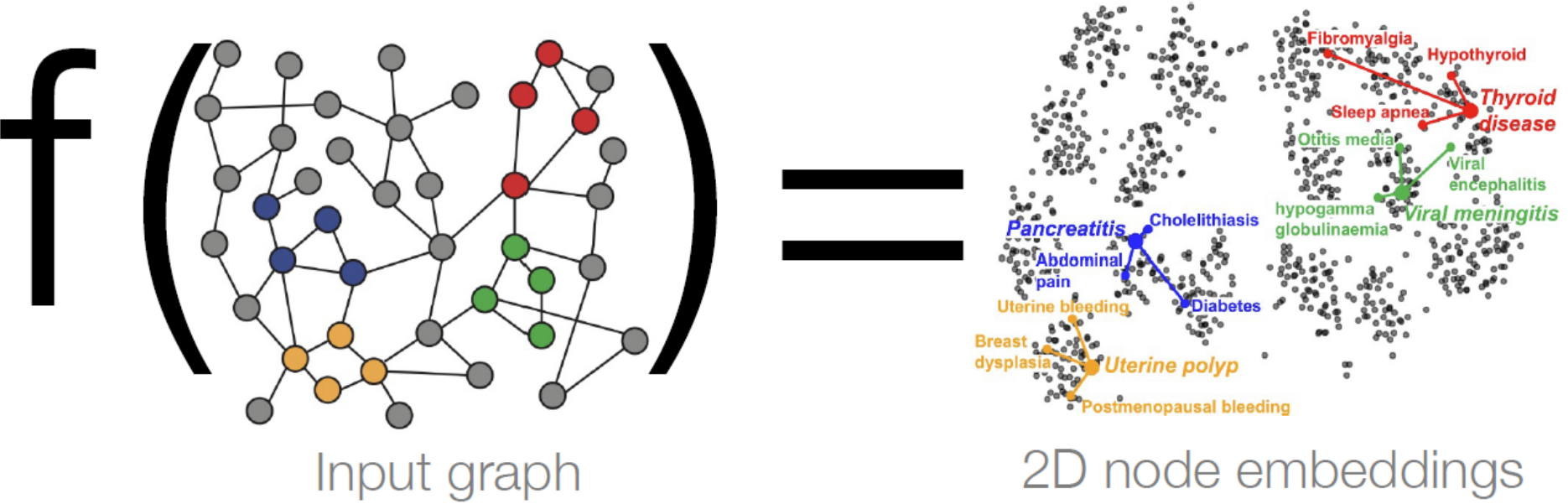
图上的数据挖掘——图表示学习

- 目标：实现图中机器学习得到的高效任务无关的特征



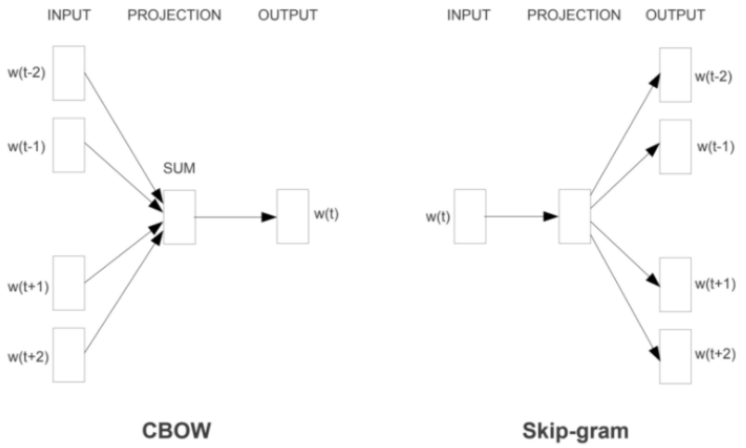
图上的数据挖掘——图表示学习

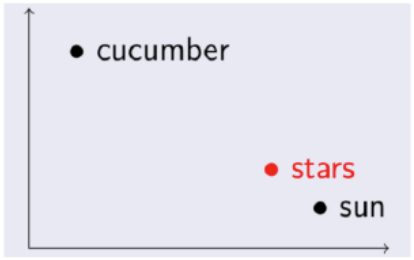
- 算法试图保留原始图中的初始结构。从本质上讲，图中相似的节点在嵌入空间中将产生相似的嵌入表示



“Shallow” Encoding

Word2vec for NLP



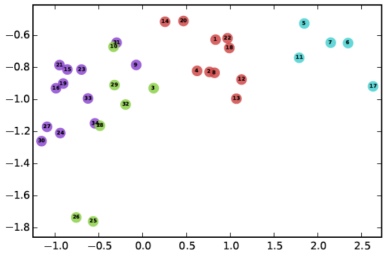


$$\|\phi(\text{sun}) - \phi(\text{stars})\| < \|\phi(\text{cucumber}) - \phi(\text{stars})\|$$

Node embedding

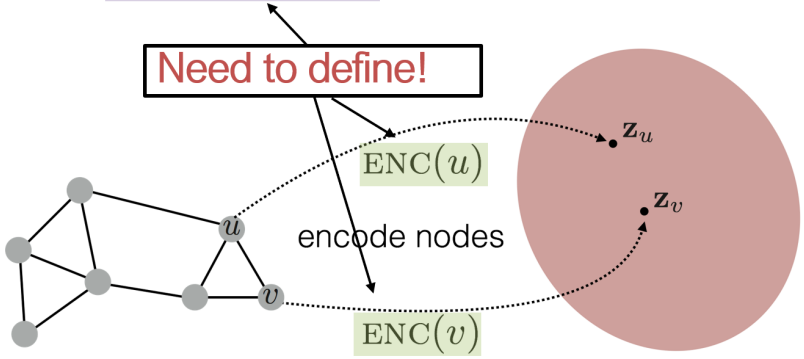


Input



Output

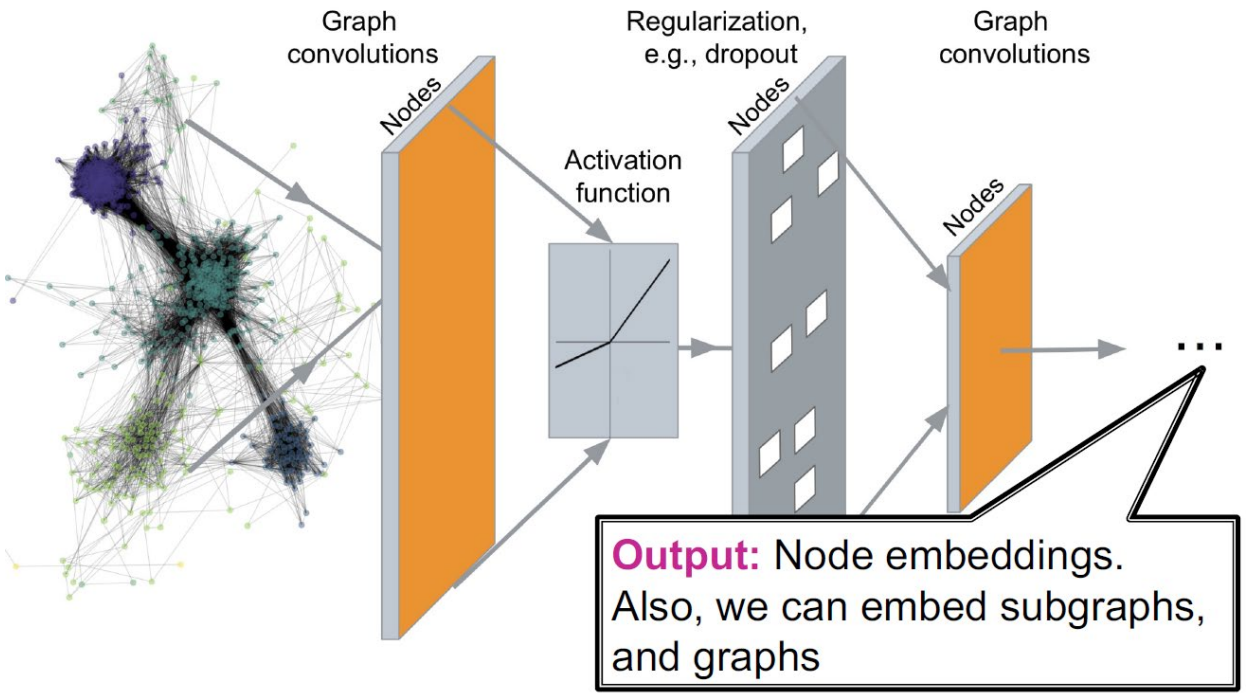
Goal: $\text{similarity}(u, v) \approx \mathbf{z}_v^\top \mathbf{z}_u$



From “Shallow” to “Deep”

- 基于图神经网络（GNNs）的深度学习方法：

$$\mathrm{ENC}(v) =$$ **multiple layers of non-linear transformations based on graph structure**



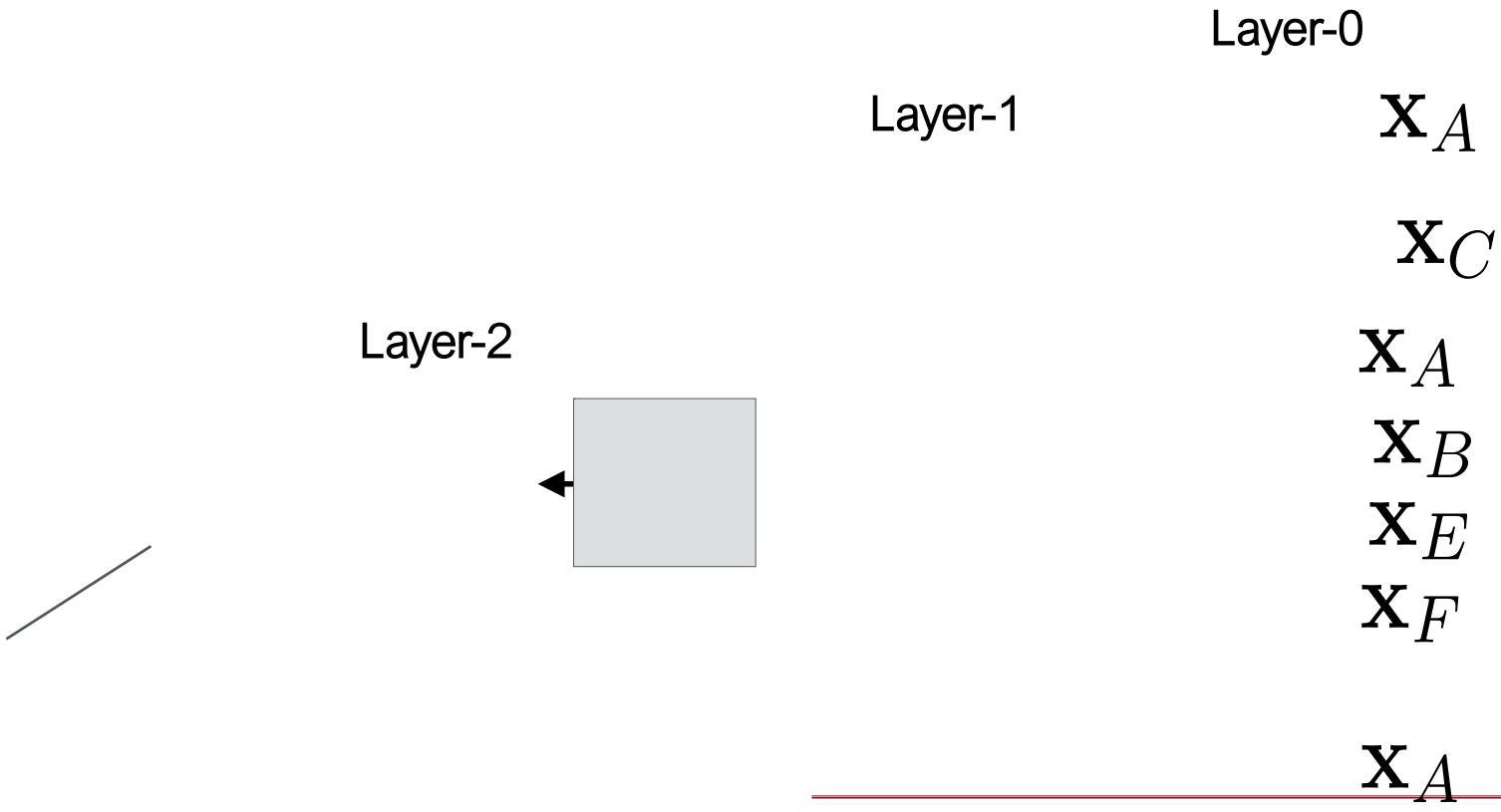
Neighborhood Aggregation

- **核心思想：**基于局部邻域生成节点嵌入表示。
- **直观理解**: 节点通过神经网络从其邻居节点聚合信息。



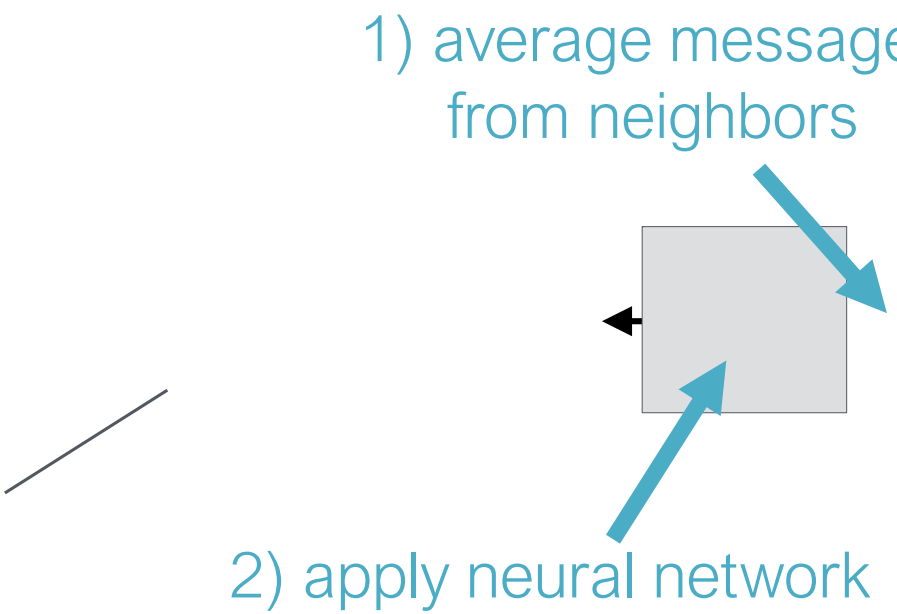
Neighborhood Aggregation

- 每个层的节点都有嵌入表示
- 模型可以是任意深度的
- 节点u的“第 0 层”嵌入是其输入特征，**i.e. x_u**



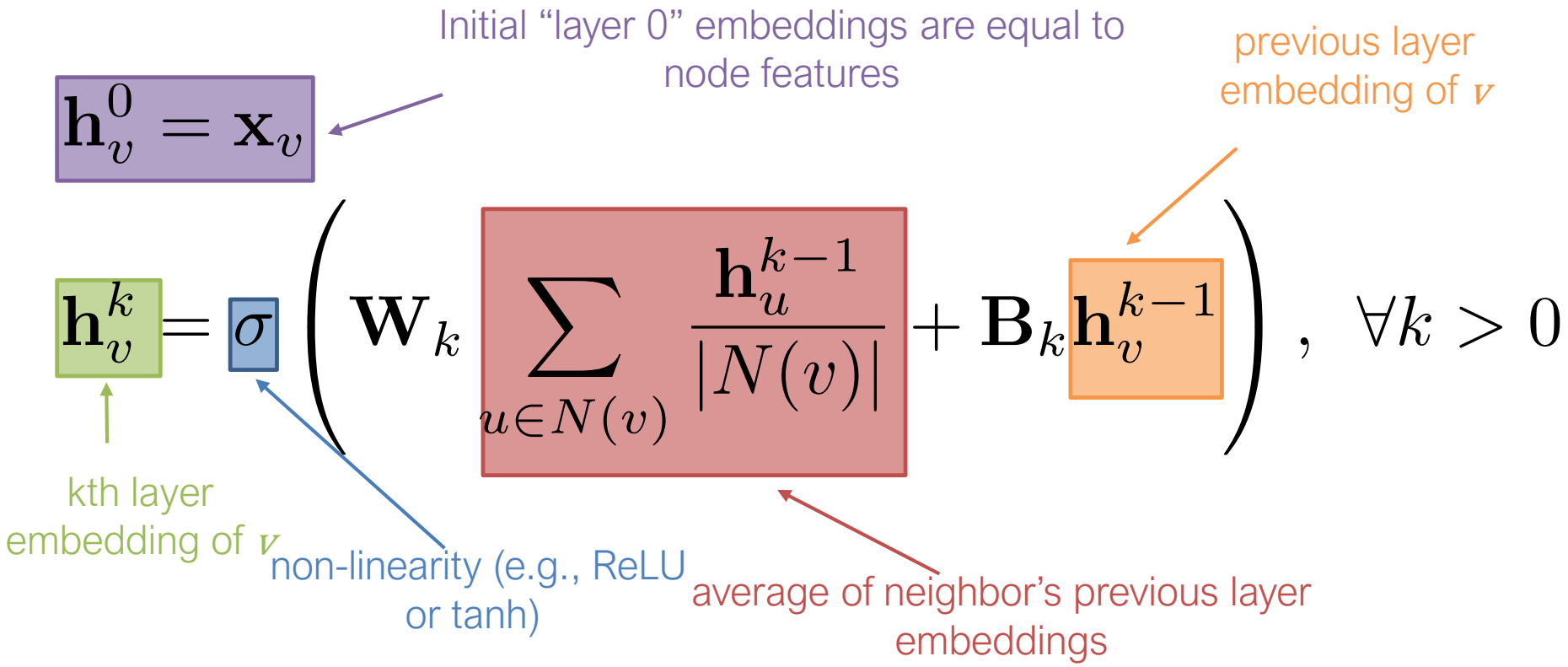
Neighborhood Aggregation

- 基本方法：对邻居信息求平均并应用神经网络



The Math

- 基本方法：对邻居信息求平均并应用神经网络



Graph Convolutional Networks

Basic Neighborhood Aggregation

$$\mathbf{h}_v^k = \sigma\left(\mathbf{W}_k \sum_{u \in N(v)} \frac{\mathbf{h}_u^{k-1}}{|N(v)|} + \mathbf{B}_k \mathbf{h}_v^{k-1}\right)$$

VS.

GCN Neighborhood Aggregation

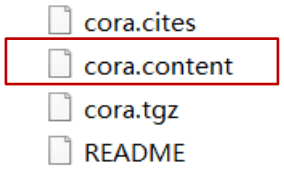
$$\mathbf{h}_v^k = \sigma\left(\mathbf{W}_k \sum_{u \in N(v) \cup v} \frac{\mathbf{h}_u^{k-1}}{\sqrt{|N(u)||N(v)|}}\right)$$

same matrix for self and neighbor embeddings

per-neighbor normalization

图表示学习的应用示例

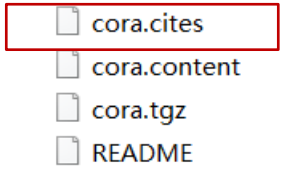
- 图节点分类任务(node classification)
- 数据集Cora
  - —2708条数据，7个分类
  - —Cora.content的数据格式
    - • <paper_id> <word_attributes>+ <class_label>




```
336   0 0 0 
```

图表示学习的应用示例

- Cora.cites是引用关系用于构建邻接矩阵
  - —Word_attributes是可选特征
  - —如使用的话作为dense embedding作为可学习参数



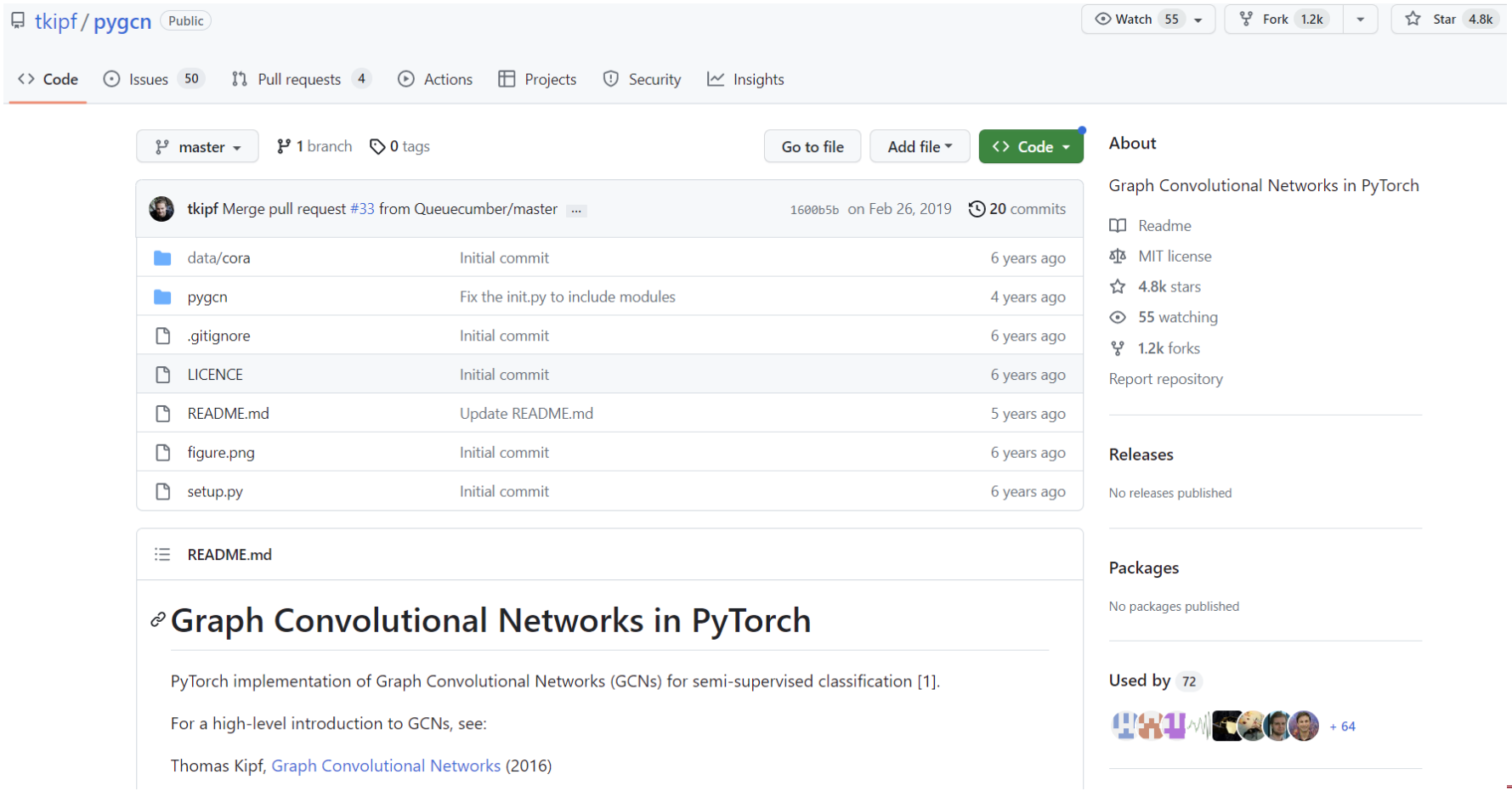

```
35  1033
35  103482
35  103515
35  1050679
35  1103960
35  1103985
35  1109199
35  1112911
35  1113438
35  1113831
35  1114331
35  1117476
35  1119505
35  1119708
35  1120431
35  1123756
35  1125386
35  1127430
35  1127913
35  1128204
35  1128227
35  1128314
35  1128453
```

图表示学习的应用示例

- 数据集按照8：2划分为训练集和测试集
- 训练和预测
  - 可以从训练集中抽取10%作为验证集，剩下90%作为真正的训练集
  - 利用图卷积网络训练节点分类模型（GCN+MLP分类器，CE loss，acc作为评估指标，GCN为2-3层）

图表示学习的应用示例

- 参考内容
  - https://github.com/tkipf/pygcn



tkipf / pygcn Public

Watch 55 Fork 1.2k Star 4.8k

Code Issues 50 Pull requests 4 Actions Projects Security Insights

master 1 branch 0 tags Go to file Add file Code

tkipf Merge pull request #33 from Queuecumber/master ... 1600b5b on Feb 26, 2019 20 commits

data/cora	Initial commit	6 years ago
pygcn	Fix the init.py to include modules	4 years ago
.gitignore	Initial commit	6 years ago
LICENCE	Initial commit	6 years ago
README.md	Update README.md	5 years ago
figure.png	Initial commit	6 years ago
setup.py	Initial commit	6 years ago

README.md

Graph Convolutional Networks in PyTorch

PyTorch implementation of Graph Convolutional Networks (GCNs) for semi-supervised classification [1].

For a high-level introduction to GCNs, see:

Thomas Kipf, Graph Convolutional Networks (2016)

About

Graph Convolutional Networks in PyTorch

Readme

MIT license

4.8k stars

55 watching

1.2k forks

Report repository

Releases

No releases published

Packages

No packages published

Used by 72

+ 64